Chapitre 1 : Introduction au langage C

IN323 - Programmation en langage C++

Elisa HENRION-GUENEAU

IPSA - Site Toulouse

AERO3 - 2025/2026

Intro

Introduction

Le C est un langage de programmation :

- Impératif, structuré et compilé.
- Créé au début des années 1970
- Lié au développement du système d'exploitation UNIX
- Influent et massivement utilisé, notamment pour la programmation système, les systèmes embarqués, etc...
- Qui a servi de base au C++ : le C++ reste compatible avec le C, mais apporte le support de la Programmation Orientée Objet et une bibliothèque standard (STL) beaucoup plus riche.

Programme C minimal

Un programme C minimal est structuré comme suit :

```
#include <stdio.h>

int main() {
    /* Le programme commence
    son execution ici */
    printf("Hello World !\n");
    return 0; // fin reussie du programme
}
```

Explications

- `include <stdio.h>` : directive de préprocesseur qui inclut la bibliothèque standard d'entrée/sortie (`stdio.h`). Nécessaire ici pour utiliser la fonction `printf`.
- `int main()` : Fonction principale de tout programme C. Le programme commence toujours son exécution ici. Elle doit retourner un entier
- `{ ... }` : Définition du bloc de code de la fonction
- `printf("Hello World !");` : Affichage d'une chaîne de caractère avec retour à la ligne

La compilation

Contrairement aux langages interprétés (ex : Python), le C est un langage **traduit** intégralement en langage machine avant exécution.
Code Source (.c) → Compilation → Fichier exécutable

- **Étape 1 : Préprocesseur** : Nettoyage
- **Étape 2 : Compilation & Assemblage**
  Traduction du code en code objet (binaire non exécutable, `.o`).
- **Étape 3 : Édition de liens (Linker)**
  Rassemblement des fichiers objets et des bibliothèques pour créer le fichier exécutable.

Exemple en ligne de commande (via GCC)

```
gcc main.c -o mon_programme
```

Variables, opérateurs

Variables et Types

En C, une variable doit être **déclarée avec un type** spécifique avant d'être utilisée.

Type	Description	Taille	Affichage
char	caractère	1 octet	%c
int	entier	4 octets	%d ou %i
short (int)	(petit) entier	2 octets	%hd
long (int)	(grand) entier	8 octets	%ld
float	nombre à virgule flottante (simple précision)	4 octets	%f
double	nombre à virgule flottante (double précision)	8 octets	%lf
void	Ensemble vide de valeurs	N/A	N/A

Déclaration et initialisation de variables

```
int age = 30; // Declaration et initialisation
float prix; // Declaration simple
prix = 19.99; // Affectation
char c = 'K';
```

Une déclaration est interprétée par le compilateur comme une demande de réservation de mémoire.

Le type de la variable permet au compilateur de savoir combien d'octets il doit lui réserver.

Affichage de variables

```
char lettre = 'A';
int entier = 42;
short petit = 10;
long grand = 1234567890L;
float pi = 3.14159f;

printf("Char : %c \n", lettre);
printf("Int  : %d \n", entier);
printf("Short: %hd \n", petit);
printf("Long : %ld \n", grand);
printf("Float: %.2f \n", pi);   // 2 chiffres
    apres la virgule
```

Lecture de données : Fonction scanf

Pour chaque donnée lue il faut préciser :

- Le format de lecture
- L'**adresse de la variable** qui recevra la valeur lue à partir du clavier (**&**)

```
#include <stdio.h>
int main (){
    int i, j;
    float a;
    printf ("\n Saisir deux entiers : ") ;
    scanf ("%d%d", &i, &j);
    printf ("\n Saisir un flottant : ") ;
    scanf ("%f", &a);
    printf ("Valeurs lues : %d %d %f", i, j, a);
    return 0 ;
}
```

Attention au "piège" du scanf

Lorsqu'on saisit un nombre, on appuie sur **Entrée**. Le caractère `'\n'` reste "bloqué" dans la mémoire tampon (le buffer).

Le problème

Si on demande une saisie de caractère (`%c`) juste après, le programme lira le `'\n'` restant au lieu d'attendre votre saisie.

→ Ajouter un **espace** avant le `%c` permet d'ignorer tous les résidus (espaces, retours à la ligne) :

```
int age;
char initiale;
printf("Age : ");
scanf("%d", &age);
printf("Initiale : ");
scanf(" %c", &initiale);
```

Constantes

Il est possible de définir une constante symbolique en utilisant `#define`

```
#include <stdio.h>
#define MAX 50

int main (){
    for (int i = 0; i <= MAX; i++){
        printf ("\n i = %d", i);
    }
    return 0;
}
```

Opérateurs

- Opérateurs **arithmétiques** : +, -, *, /, %
- Opérateurs de **comparaisons** : ==, !=, >, >=, <, <=
- Opérateurs **logiques** : || (ou), && (et), ! (not)
- Opérateur **conditionnel** : exp1 ?exp2 :exp3
  → vaut exp2 si exp1 vaut 1 (vrai), exp2 sinon

Struct. de contrôle

Conditions : if, else if, else

```
int note = 15;

if (note >= 10) {
    printf("Admis !\n");
} else if (note >= 8) {
    printf("Rattrapage.\n");
} else {
    printf("Echec.\n");
}
```

Choix multiples : switch

- Instruction `switch` : offrir plusieurs choix e fonction de la valeur d'une variable

```
int i;
printf("Donner le numero du jour :");
scanf("%d", &i);
switch(i){
    case 1 : printf("Lundi");
             break;
    case 2 : printf("Mardi");
             break;
    ...
    case 7 : printf("Dimanche");
             break;
    default : printf("Numero incorrect");
}
```

Boucles : while, for, do-while

- **Boucle `while`** (tant que la condition est vraie) :

```
int i = 0;

while (i < 5) {
    printf("Compteur : %d\n", i);
    i++; // i = i + 1;
}
```

Boucles : while, for, do-while

- **Boucle `for`** (idéale pour un nombre d'itérations connu) :

```
for (int j = 0; j < 3; j++) {
    printf("Tour numero %d\n", j);
}
```

- **Boucle `do-while`** (garantit que la boucle est exécutée au moins une fois avant que la condition ne soit vérifiée).

```
int k = 0;
do {
    printf("Execution numero %d\n", k);
    k++;
} while (k < 3); // verifiee apres la 1ere
   execution
```

Boucles : while, for, do-while

Chacune des boucles proposées par le langage C peut être utilisée pour exprimer n'importe quel traitement répétitif.

- Passage de `while` à `for` :

```
Initialisations ;
while (Condition)
{
        Instruction ;
        Instruction ;
        ............
        Changement ;
}
```

⇒

```
for (Initialisations ; Condition ; Changement)
{
        Instruction ;
        Instruction ;
        ............
}
```

- Passage de `while` à `do-while`

```
Initialisation ;
while (Condition)
{
        Corps
}
```

⇒

```
Initialisation ;
if (Condition)
        do
        {
                Corps
        } while (Condition) ;
```

Boucles : while, for, do-while

Chacune des boucles proposées par le langage C peut être utilisée pour exprimer n'importe quel traitement répétitif.

- Passage de `do-while` à `while`

Initialisation ;
do
{
Corps
} while (Condition) ;
	Initialisation ;
	Corps
	while (Condition)
	{
	Corps
	}

- Passage de `for` à `while`

for (Initialisation ; Condition ; Changement)
{
Corps
}
	Initialisation ;
	while (Condition)
	{
	Corps
	Changement ;
	}

Branchement non conditionnel : break, continue

- Instruction `break` : sortir de la boucle

```
int i = 1;
while (i <= 10){
    if (i == 5) {
        break;
    }
    printf ("i = %d", i);
    i++;
}
```

Attention :

Si on peut exprimer la condition d'arrêt dans la boucle, c'est mieux.
→ `break` uniquement pour cas exceptionnels / interruptions qu'on ne peut pas prévoir au début de l'itération.

Branchement non conditionnel : break, continue

- Instruction `continue` : passer à l'itération suivante

```
for (int i = 1; i <= 5; i++) {
    if (i == 3) {
        printf("--- Valeur 3 ignoree ---\n");
        continue; // Saute le printf
    }
    printf("Traitement du numero %d\n", i);
}
```

Attention :

`continue` brise la lecture naturelle de l'algo → privilégier un `if/else` .
Dans une boucle `while`, un `continue` mal placé peut sauter l'incrémentation (ex : `i++`) et provoquer une boucle infinie.

Fonctions

Définir ses propres fonctions

Une fonction permet de segmenter le code en blocs réutilisables.

```
// Prototype (souvent dans un fichier .h)
int addition(int a, int b);

int main() {
    int res = addition(5, 3);
    return 0;
}

// Definition
int addition(int a, int b) {
    return a + b;
}
```

- **Type de retour** : `void` si la fonction ne renvoie rien.
- **Prototype** : Indique au compilateur que la fonction existe avant de l'appeler.

Les procédures (type void)

Si une fonction ne doit pas renvoyer de résultat (ex : juste un affichage), on utilise le type `void`.

```
void direBonjour() {
    printf("Bonjour !\n");
}

int main() {
    direBonjour();
    return 0;
}
```

Pointeurs

Introduction

Lorsque l'on manipule une variable, sa valeur est stockée dans la mémoire.

Mémoire :

Constituée d'octets, qui sont identifiés de manière unique par un numéro appelé **adresse**.

Quand on déclare une variable, le compilateur lui associe un ou plusieurs octets consécutifs (en fonction de son type) où sera stockée sa valeur.

→ Pour retrouver la valeur d'une variable, il suffit de connaître son type et l'adresse du premier octet où est stockée sa valeur.

Adresse d'une variable

On obtient l'adresse d'une variable via l'opérateur d'adresse &

Adresse :

L'adresse d'un objet étant un numéro d'octet, c'est donc un entier. Son format dépend de l'architecture de l'ordinateur (32 ou 64 bits)

Pour pouvoir manipuler les adresses de variables, le langage C met à notre disposition des variables spéciales appelées **"pointeurs"**.

Pointeur

Définition :

Un pointeur est un objet dont la valeur est égale à l'adresse d'un autre objet.
La déclaration d'un pointeur se fait selon le modèle suivant :

```
type *nom_pointeur ;
```

Notations :

- Le type **T *** : type pointeur vers une variable de type T
- la notation **&v** : adresse de la variable v
- Si **p** est un pointeur de type **T ***, la valeur pointée est notée ***p** (valeur de type T)
- un pointeur peut valoir NULL (valeur spéciale signifiant que le pointeur ne pointe sur rien)

Pointeur

Exemple :

```
int n = 350;
int *p1 = NULL;
p1 = &n; // p1 recoit l'@ de n
printf("Valeur n : %d\n", n); //350
printf("Valeur p1 : %p\n", p1); //@ de n
printf("Valeur pointee par p1 : %d\n", *p1); //350
printf("Adresse de n : %p\n", &n); //<=> p1
printf("Adresse de p1 : %p\n", &p1); //@ de p1
```

- p1 pointe sur n car il contient son adresse
- n et *p1 sont équivalents
- &n et p1 sont équivalents

Passage par valeur vs adresse

- **Par valeur :** La fonction travaille sur une **copie**. L'original reste inchangé.
- **Par adresse :** On passe un **pointeur**. La fonction peut modifier directement la variable d'origine.

Passage par adresse

```
#include <stdio.h>

void swap(int *a, int *b) {
    int tmp = *a;
    *a = *b;
    *b = tmp;
}
int main() {
    int v1 = 10;
    int v2 = 100;
    printf("Avant : v1 = %d, v2 = %d\n", v1, v2);
    swap(&v1, &v2);
    printf("Apres : v1 = %d, v2 = %d\n", v1, v2);
    return 0;
}
```

Tableaux

Les Tableaux Statiques

Un tableau regroupe des variables de même type, stockées de manière contiguë (côte à côte) en mémoire.

- **Déclaration :** `type nom[taille_fixe];`
- **Initialisation :**

```
int t1[5]; // Reserve l'espace pour 5 entiers
int t2[3] = {12, 15, 18}; // Taille connue
int t3[] = {1, 2, 3}; // Taille deduite (3)
```

Limites du statique

- La taille doit être une constante, pas une variable.
- Impossible de redimensionner en cours de route.

Le lien Tableaux et Pointeurs

En C, le nom d'un tableau est une **adresse constante** vers son premier élément.

- `tab` est strictement équivalent à `&tab[0]`.
- Quand on passe un tableau à une fonction, on passe en réalité un **pointeur**.

```
int tab[3] = {10, 20, 30};
int *p = tab; // p pointe sur le 1er element : 10
printf("%d", tab[1]); // Acces classique : 20
printf("%d", *(p + 1)); // Acces via pointeur : 20

int mat[3][3] = {{1,2,3},{4,5,6},{7,8,9}};
int (*q)[3] = mat; // q pointe sur {1,2,3}
printf("%d", mat[1][2]); //Acces classique : 6
printf("%d", *(*(q + 1)+2)); //Acces pointeur : 6
```

Copier un tableau : Le piège du "="

Attention :

On ne peut pas copier un tableau en écrivant simplement `tab2 = tab1;` car le nom d'un tableau est une **adresse**

```
int src[3] = {10, 20, 30};
int dest[3];
for (int i = 0; i < 3; i++) {
    dest[i] = src[i]; // COPIE MANUELLE
}
```

Alternative rapide : memcpy (<string.h>)

Pour les gros tableaux, on utilise `memcpy(dest, src, taille_en_octets)` :

```
memcpy(dest, src, 3 * sizeof(int));
```

Cas particulier : Les chaînes de caractères

En C, une "String" n'existe pas en tant que type natif.
C'est un **tableau de char** qui se termine par un caractère spécial : `'\0'` (code ASCII 0).

```
char nom[] = "C++";
// En memoire : ['C', '+', '+', '\0'] (4 octets)
```

`#include <string.h>` pour les manipuler :

- `strlen(s)` : longueur de la chaîne (sans le \0)
- `strcmp(s1, s2)` : compare deux chaînes *(renvoie 0 si =)*
- `strcpy(dest, src)` : copie une chaîne *(attention à la taille)*
- `strcat(s1, s2)` : concatène deux chaînes

Besoin d'allocation dynamique

Comment faire si l'on ne connaît la taille du tableau qu'**après** le lancement du programme ? (ex : saisie utilisateur).

On utilise le **Tas (Heap)** : une zone mémoire vaste où l'on peut réserver et libérer des blocs à la demande.

Les deux fonctions clés (<stdlib.h>) :

- `malloc(taille)` : "Memory Allocate", réserve un bloc d'octets.
- `free(pointeur)` : Libère le bloc pour qu'il soit réutilisable.

Mise en œuvre du Tableau Dynamique

```
#include <stdlib.h>
int main() {
    int n;
    printf("Nombre d'elements : ");
    scanf("%d", &n);

    // ETAPE 1 : Allocation
    int *tab = malloc(n * sizeof(int));
    if (tab == NULL) { // Securite
        return 1;
    }
    //... ETAPE 2 : TRAITEMENT CLASSIQUE ...
    // ETAPE 3 : Liberation de la memoire
    free(tab);
    return 0;
}
```

Structures

Les différents types de variables

Les types simples

Entier (int), Nombre à virgule (float), caractère (char)

Les types composés - homogènes

Tableau, Chaîne de caractères (tableau de char)

Les tableaux et les chaînes de caractères sont des types composés **homogènes** car ils regroupent de éléments de même type.

Structures

Les types composés - hétérogènes :

Les structures permettent de créer des types composés hétérogènes

Objectif d'une structure :

Regrouper plusieurs valeurs (éventuellement de types différents) sous un même type.

En C :

```
struct nomStructure {
    type1 champ1;
    type2 champ2;
    ...
};
```

Exemple

Création d'un type Etudiant

Un étudiant est défini par son prénom, son âge et sa moyenne.

```
struct Etudiant {
    char nom[50];
    int age;
    float moyenne;
};
```

Exemple

Création d'un Etudiant

```
struct Etudiant e1  = {"Alice", 20, 15.5};
```

Utilisation d'un Etudiant

```
e1.age = 21;
printf("%s a %d ans.\n", e1.nom, e1.age);
// Alice a 21 ans.
```

Simplifier l'écriture : typedef

Pour éviter de réécrire le mot-clé `struct` à chaque fois, on définit un alias avec `typedef`.

```
typedef struct {
    char prenom[50];
    int age;
    float moyenne;
} Etudiant; // Le nom du type est a la fin

int main() {
    Etudiant e1 = {"Alice", 21, 15.5};
    return 0;
}
```

Pointeurs et Structures : l'opérateur ->

En C, lorsqu'on manipule l'adresse d'une structure (via un pointeur), on n'utilise pas le point mais l'opérateur **flèche** (->).

```
Etudiant e1 = {"Alice", 21, 15.5};
Etudiant *p = &e1; // contient l'adresse de e1

// Acces aux champs via le pointeur :
p->age = 22;
printf("%s a %d ans.\n", p->prenom, p->age);
```

Équivalence

`p->age` est un raccourci d'écriture pour `(*p).age`.
C'est la syntaxe standard utilisée par tous les programmeurs C.

Structures et Fonctions : Passage par adresse

Pour modifier une structure dans une fonction, on passe son **adresse**.

```
void feterAnniversaire(Etudiant *e) {
    e->age++; // Modifie directement l'original
}
int main() {
    Etudiant e1 = {"Alice", 21, 15.5};
    feterAnniversaire(&e1); // On envoie l'adresse
    printf("Age : %d", e1.age); // Affiche 22
    return 0;
}
```

Performance

Passer un pointeur (`Etudiant *`) ne copie que l'adresse, peu importe la taille de la structure. Passer la structure entière (`Etudiant`) copierait tous ses champs, ce qui est lent.

Allocation dynamique de structure

On peut combiner les structures avec `malloc` pour créer des objets à la demande pendant l'exécution.

```
// Allocation d'un etudiant
Etudiant *e = malloc(sizeof(Etudiant));

if (e != NULL) {
    strcpy(e->prenom, "Bob");
    e->age = 20;
    e->moyenne = 12.5;

    printf("%s cree dynamiquement.\n", e->prenom);
    free(e); // Ne pas oublier de liberer
}
```

Biblio

Mathématiques : math.h

```
#include <math.h>
```

Fonction	Rôle
abs(x)	valeur absolue d'un entier
fabs(x)	valeur absolue d'un flottant
cos, sin, tan(x) acos, asin, atan(x)	fonctions trigonométriques
log, exp, sqrt(x)	logarithme, exponentielle, racine carrée
pow(x,y)	x puissance y
ceil(x)	+ petit entier >= x
floor(x)	+ grand entier <= x
round(x)	arrondir à l'entier le plus proche

Analyse de caractères : ctype.h

Cette bibliothèque permet de tester ou de transformer des caractères sans manipuler manuellement les codes ASCII.

Fonction	Rôle (renvoie vrai si...)
`isdigit(c)`	... `c` est un chiffre (0-9)
`isalpha(c)`	... `c` est une lettre (A-Z, a-z)
`isspace(c)`	... `c` est un espace ou tabulation
`isupper(c)` / `islower(c)`	... `c` est en majuscule / minuscule

Transformation

- `toupper(c)` : convertit c en majuscule.
- `tolower(c)` : convertit c en minuscule.

Temps : time.h

```
#include <time.h>
```

La fonction `time(NULL)` retourne un entier qui représente le temps écoulé depuis le 01/01/1970 (en secondes)

```
int temps;
temps = time(NULL);
printf ("Temps : %d secondes", temps);
```

Aléatoire : stdlib.h

```
#include <stdlib.h>
```

La fonction `rand()` renvoie un entier (pseudo) aléatoire

```
int n = rand();
printf("Nombre : %d", n);
```

Si on lance plusieurs fois, on obtiendra toujours le même nombre, parce que `rand()` utilise une formule fixe basée sur une "graine" (seed) qui vaut 1 par défaut.

Pour avoir un résultat différent à chaque exécution, on initialise la graine avec l'heure actuelle via `srand(time(NULL))`

```
srand(time(NULL));
printf("Nombre 1 : %d\n", rand());
printf("Nombre 2 : %d\n", rand());
```